

PROGRAMMING IN HASKELL



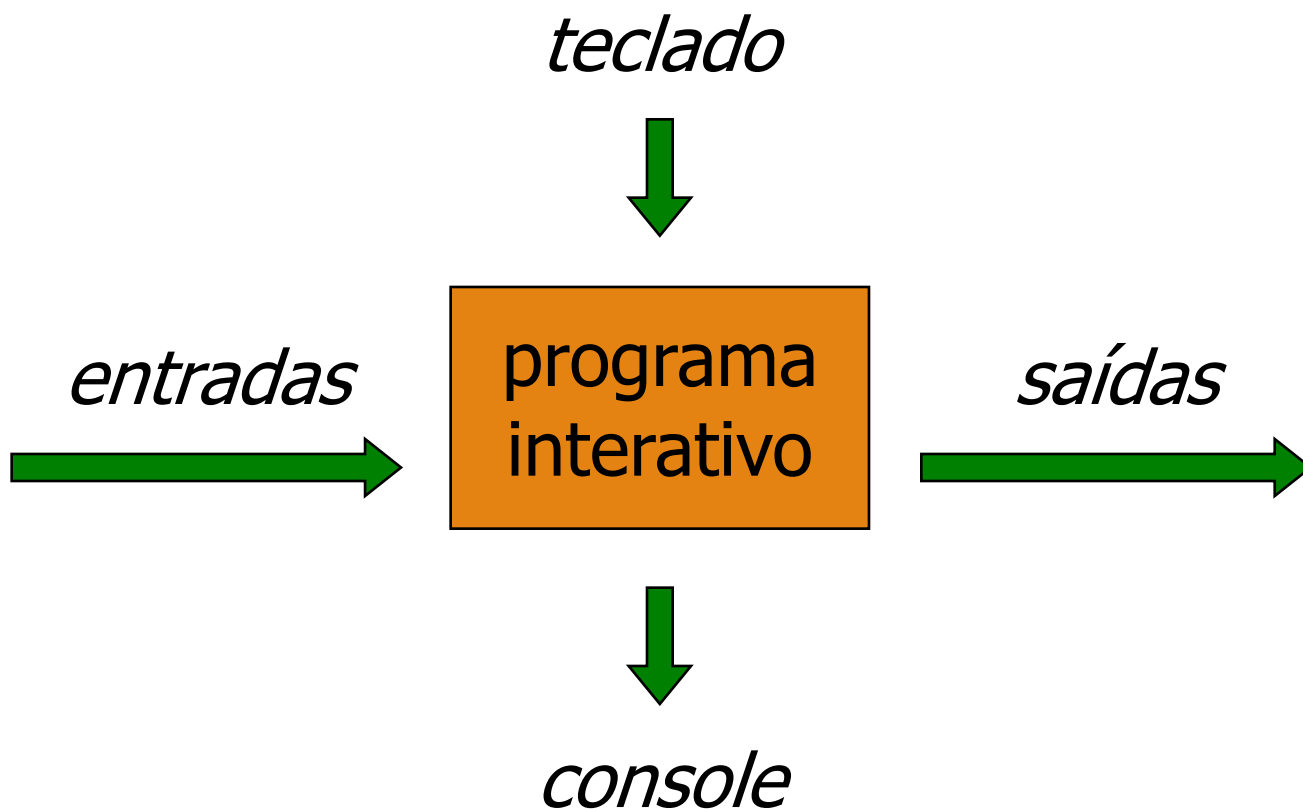
Capítulo 10 - Programação Interativa

Introdução

Até o momento, nós vimos como Haskell pode ser utilizado para escrever programas em batch que tomam todas entradas no início e produzem todas saídas ao final.



Entretanto, também gostaríamos de utilizar Haskell para escrever programas interativos que lêem do teclado e escrevem no console, enquanto estão executando.



O Problema

Programas em Haskell são funções matemáticas puras:

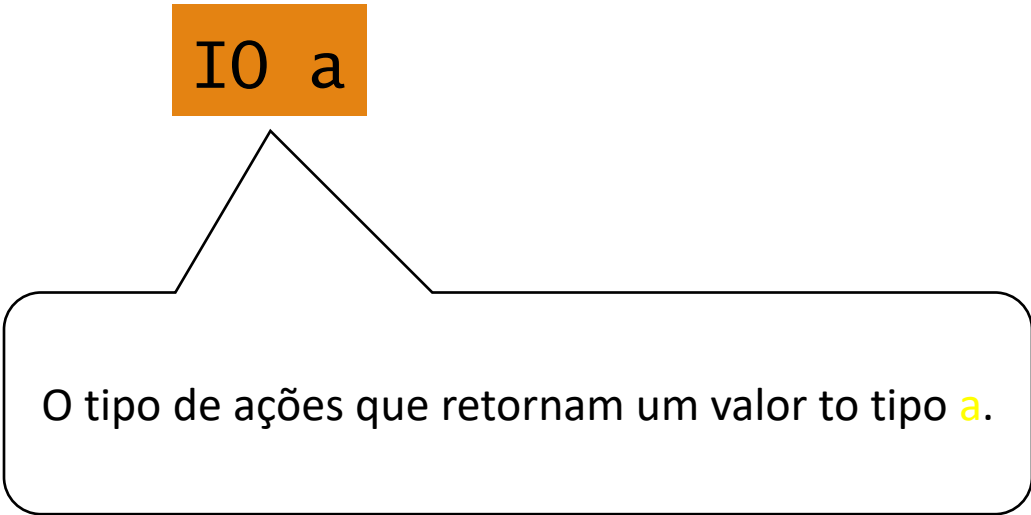
- Programas em Haskell não possuem efeitos colaterais.

Entretanto, ler do teclado e escrever no console são efeitos colaterais:

- Programas interativos possuem efeitos colaterais.

A Solução

Programas interativos podem ser desenvolvidos em Haskell através do uso de tipos que permitem a distinção entre expressões puras de ações impuras que podem envolver efeitos colaterais.



IO a

O tipo de ações que retornam um valor to tipo **a**.

Por exemplo:

IO Char

O tipo de ação que retorna um caractere.

IO ()

O tipo de ação que não retorna valores.

Nota:

- () é o tipo para tuplas sem componentes.

Ações Básicas

A biblioteca padrão provê vários tipos de ações, incluindo as seguintes três ações primitivas:

- A ação getChar lê um caractere do teclado, ecoa no console, e retorna o caractere como resultado:

```
getChar :: IO Char
```

- A ação putChar *c* escreve o caractere *c* no console, e não retorna um valor:

```
putChar :: Char → IO ()
```

- A ação return *v* simplesmente retorna o valor *v*, sem realizar qualquer interação:

```
return :: a → IO a
```


Sequência de Ações

Uma sequência de ações pode ser combinada em uma única ação composta através da notação do.

Por exemplo:

```
act :: IO (Char,Char)
act = do x ← getChar
        getChar
        y ← getChar
        return (x,y)
```

Primitivas Derivadas

- Ler uma string do teclado:

```
getLine :: IO String
getLine = do x ← getChar
            if x == '\n' then
                return []
            else
                do xs ← getLine
                   return (x:xs)
```

- Escrever uma string no console:

```
putStr :: String → IO ()  
putStr []      = return ()  
putStr (x:xs) = do putChar x  
                  putStr xs
```

- Escrever uma string e mover para nova linha:

```
putStrLn :: String → IO ()  
putStrLn xs = do putStr xs  
                putChar '\n'
```

Exemplo

Podemos definir uma ação que solicita a entrada de uma string e mostra o seu tamanho:

```
strlen :: IO ()
strlen = do putStr "Enter a string: "
           xs ← getLine
           putStr "The string has "
           putStr (show (length xs))
           putStrLn " characters"
```

Por exemplo:

```
> strlen
```

```
Enter a string: Haskell
```

```
The string has 7 characters
```

Nota:

- Avaliar uma ação resulta na execução de seus efeitos colaterais, com o resultado final sendo descartado.

Jogo da Forca

Considere a seguinte versão do jogo da forca (hangman):

- Um jogador secretamente fornece uma palavra.
- O outro jogador tenta deduzir a palavra, através de múltiplas tentativas.
- Para cada tentativa, o computador indica quais letras da palavra secreta ocorrem na palavra fornecida na tentativa.
- O jogo termina quando for descoberta a palavra secreta.

Adotou-se um mecanismo top down para a implementação do jogo hangman em Haskell, iniciando como apresentado:

```
hangman :: IO ()
hangman = do putStrLn "Think of a word: "
            word ← sgetLine
            putStrLn "Try to guess it:"
            play word
```

A ação sgetline lê uma linha de texto do teclado, ecoando cada caractere como um traço:

```
sgetline :: IO String
sgeline = do x ← getCh
           if x == '\n' then
             do putChar x
                return []
           else
             do putChar '-'
                xs ← sgetline
                return (x:xs)
```


A ação getCh lê um único caractere do teclado, sem ecoar no console:

```
import System.IO

getCh :: IO Char
getCh = do hSetEcho stdin False
           x ← getChar
           hSetEcho stdin True
           return x
```

A função play é o laço principal, que requisita e processa as tentativas até o final do jogo.

```
play :: String → IO ()
play word =
    do putStr "? "
       guess ← getLine
       if guess == word then
           putStrLn "You got it!"
       else
           do putStrLn (match word guess)
              play word
```

A função match indica quais caracteres em uma string ocorrem na segunda string:

```
match :: String → String → String
match xs ys =
    [if elem x ys then x else '-' | x ← xs]
```

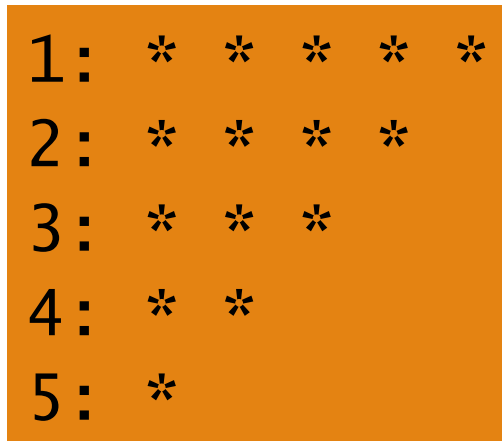
Por exemplo:

```
> match "haske11" "pasca1"
"-as--11"
```

Exercício

Implemente o jogo de nim em Haskell, onde as regras do jogo são:

- O tabuleiro consiste em 5 linhas de estrelas:



The diagram shows a Nim game board with 5 rows of stars. The stars are arranged in a descending staircase pattern from top to bottom.

1:	*	*	*	*	*
2:	*	*	*	*	
3:	*	*	*		
4:	*	*			
5:	*				

- Dois jogadores se revezam removendo uma ou mais estrelas do final de uma única linha.
- O vencedor é o jogador que remover a(s) última(s) estrela(s) do tabuleiro.

Dica:

Represente o tabuleiro como uma lista de 5 inteiros indicando o número restante de estrelas em cada linha. Por exemplo, o tabuleiro inicial seria [5,4,3,2,1].